# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Database Management Systems**
**COURSE CODE: CSA0504**

---

### COURSE OUTCOME COVERED

**CO3:** *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

---

| **QUESTIONS** | | | Total Marks: 50 |
|---|---|---|---|
| **Q.No** | **Question** | **Bloom's Level** | **Marks** |
| **1** | For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys. | BL2 – Understand | **5** |
| **2** | Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example. | BL3 – Apply | **5** |
| **3** | Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies. | BL3 – Apply | **5** |
| **4** | Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies. | BL3 – Apply | **5** |
| **5** | Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF. | BL3 – Apply | **5** |
| **6** | Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm. | BL3 – Apply | **5** |

| | | | |
|---|---|---|---|
| **7** | Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF. | BL3 – Apply | **5** |
| **8** | Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form). | BL3 – Apply | **5** |
| **9** | Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF. | BL2 – Understand | **5** |
| **10** | Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation. | BL3 – Apply | **5** |

---

***Code of Conduct:***

*I **R.Pranathi(192525076**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** R. Pranathi

## RUBRICS FOR CALCULATION:

| Criteria | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|
| Understanding of Concepts | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design | Well-planned, systematic implementation with optimal solution | Correct implementation with minor issues | Basic implementation with errors or inefficiencies | Incorrect or incomplete implementation |
| Use of Tools | Effective and advanced use of tools/technologies with proper configuration | Appropriate use of tools with correct execution | Limited or inconsistent use of tools | Incorrect or minimal use of tools |
| Analysis of Results | Insightful analysis with accurate interpretation and validation of results | Good analysis with reasonable interpretation | Basic analysis with limited insights | Poor or incorrect analysis of results |

| | | | | |
|---|---|---|---|---|
| Documentation | Clear, well-structured documentation with diagrams, code, and results | Organized documentation with necessary details | Basic documentation with missing details | Poor or incomplete documentation |

## 1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys

**ANS:**

Given the relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

**Understand the attributes**

| Attribute | Meaning |
|---|---|
| SID | Student ID |
| SName | Student Name |
| CID | Course ID |
| CName | Course Name |
| Instructor | Course Instructor |
| Grade | Grade obtained by student |

Assume that:

- Each SID identifies exactly one student.
- Each CID identifies exactly one course and instructor.
- A student's grade is determined by the combination of student and course.

**Identify Functional Dependencies**

**FD 1: Student ID determines Student Name**

A particular student ID belongs to only one student.

$$SID \rightarrow SName$$

Example:

SID = S101 → SName = Ravi

**FD 2: Course ID determines Course Name**

Each course has a unique course ID.

$$CID \rightarrow CName$$

Example:

CID = C01 → CName = DBMS

**FD 3: Course ID determines Instructor**

Assuming each course has one instructor:

$$CID \rightarrow Instructor$$

Example:

CID = C01 → Instructor = Kumar

Therefore, we can combine the two:

$$CID \rightarrow CName, Instructor$$

**FD 4: Student ID + Course ID determines Grade**

A student's grade depends on **which course** the student took.

Therefore:

$$SID,CID \rightarrow Grade$$

For example:

SID = S101 + CID = C01 → Grade = A

**List All Main Functional Dependencies**

Therefore, the functional dependencies are:

SID→SName
CID→CName
CID→Instructor
SID,CID→Grade

Or in combined form:

SID→SName,CID→CName,Instructor,SID,CID→Grade

**Find the Candidate Key**

We need an attribute set that can determine **all attributes** of the relation.

Try:

{SID,CID}

Calculate its closure:

(SID,CID)+

Start with:

(SID,CID)+={SID,CID}

**Apply FD 1**

We know:

SID→SName

Therefore add SName.

(SID,CID)+={SID,CID,SName}

**Apply FD 2 and FD 3**

We know:

CID→CName

And

CID→Instructor

Therefore add CName and Instructor.

(SID,CID)+={SID,CID,SName,CName,Instructor}

**Apply FD 4**

We know:

SID,CID→Grade

Therefore add Grade.

(SID,CID)+={SID,SName,CID,CName,Instructor,Grade}

We have obtained **all attributes** of the relation.

Therefore:

(SID,CID) is a superkey

**Check Minimality**

Now check whether we can remove either attribute.

**Remove SID**

Try:

CID+

We get:

CID+={CID,CName,Instructor}

It cannot determine:

- SID
- SName
- Grade

Therefore, CID alone is **not a key**.

**Remove CID**
Try:
SID+
We get:
SID+={SID,SName}
It cannot determine:

- CID
- CName
- Instructor
- Grade

Therefore, SID alone is **not a key**.

Since neither SID nor CID can be removed:
(SID,CID) is a minimal superkey
Hence it is a **candidate key**.

**Functional Dependencies**
SID→SName
CID→CName
CID→Instructor
SID,CID→Grade
**Candidate Key**
Candidate Key=(SID,CID)

## 2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

## ANS:

```
mysql> create database DBMS;
Query OK, 1 row affected (0.03 sec)

mysql> use DBMS;
Database changed
mysql> CREATE TABLE Student_UNF ( SID INT, SName VARCHAR(50), Courses VARCHAR(200) );
Query OK, 0 rows affected (0.13 sec)

mysql> INSERT INTO Student_UNF VALUES (101, 'Arun', 'DBMS, OS, Python'), (102, 'Priya', 'DBMS, Java'), (103, 'Rahul', 'OS, Python');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student_UNF;
+------+-------+------------------+
| SID  | SName | Courses          |
+------+-------+------------------+
|  101 | Arun  | DBMS, OS, Python |
|  102 | Priya | DBMS, Java       |
|  103 | Rahul | OS, Python       |
+------+-------+------------------+
3 rows in set (0.02 sec)

mysql> CREATE TABLE Student_1NF ( SID INT, SName VARCHAR(50), Course VARCHAR(50), PRIMARY KEY (SID, Course) );
Query OK, 0 rows affected (0.12 sec)

mysql> INSERT INTO Student_1NF VALUES (101, 'Arun', 'DBMS'), (101, 'Arun', 'OS'), (101, 'Arun', 'Python'), (102, 'Priya', 'DBMS'), (102, 'Priya', 'Java'), (
103, 'Rahul', 'OS'), (103, 'Rahul', 'Python');
Query OK, 7 rows affected (0.06 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student_1NF;
+-----+-------+--------+
| SID | SName | Course |
+-----+-------+--------+
| 101 | Arun  | DBMS   |
| 101 | Arun  | OS     |
| 101 | Arun  | Python |
| 102 | Priya | DBMS   |
| 102 | Priya | Java   |
| 103 | Rahul | OS     |
| 103 | Rahul | Python |
+-----+-------+--------+
7 rows in set (0.00 sec)
```

**Justification for Conversion from UNF to 1NF**
The given relation is initially unnormalized (UNF) because the Courses attribute contains multiple values in a single cell, such as DBMS, OS, Python.
To convert it into 1NF:

1. Identify the repeating/multivalued attribute — Courses.
2. Separate the multiple course values into individual rows.

3. Ensure that each cell contains only one atomic value.
4. Since one student can take multiple courses, SID alone cannot uniquely identify each record.
5. Therefore, (SID, Course) is used as the composite primary key.
6. After this conversion, there are no repeating groups or multivalued attributes.

Hence, the resulting STUDENT_1NF(SID, SName, Course) relation satisfies 1NF because every attribute contains only atomic values and each record is uniquely identifiable.

## 3. Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.

**ANS:**

```
mysql> create database A1;
Query OK, 1 row affected (0.07 sec)

mysql> use A1;
Database changed
mysql> CREATE TABLE R ( A INT, B INT, C INT, D INT, E INT );
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO R VALUES (1, 10, 20, 100, 500), (2, 30, 40, 200, 600), (3, 50, 60, 300, 700);
Query OK, 3 rows affected (0.12 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM R;
+------+------+------+------+------+
| A    | B    | C    | D    | E    |
+------+------+------+------+------+
|    1 |   10 |   20 |  100 |  500 |
|    2 |   30 |   40 |  200 |  600 |
|    3 |   50 |   60 |  300 |  700 |
+------+------+------+------+------+
3 rows in set (0.05 sec)

mysql> CREATE TABLE R1 ( A INT PRIMARY KEY, B INT, C INT );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE R2 ( B INT, C INT, D INT, PRIMARY KEY (B, C) );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE R3 ( D INT PRIMARY KEY, E INT );
Query OK, 0 rows affected (0.13 sec)

mysql> INSERT INTO R1 VALUES (1, 10, 20), (2, 30, 40), (3, 50, 60);
Query OK, 3 rows affected (0.10 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM R1;
+---+------+------+
| A | B    | C    |
+---+------+------+
| 1 |   10 |   20 |
| 2 |   30 |   40 |
| 3 |   50 |   60 |
+---+------+------+
3 rows in set (0.05 sec)

mysql> INSERT INTO R2 VALUES (10, 20, 100), (30, 40, 200), (50, 60, 300);
Query OK, 3 rows affected (0.08 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM R2;
+----+----+------+
| B  | C  | D    |
+----+----+------+
| 10 | 20 |  100 |
| 30 | 40 |  200 |
| 50 | 60 |  300 |
+----+----+------+
3 rows in set (0.02 sec)

mysql> INSERT INTO R3 VALUES (100, 500), (200, 600), (300, 700);
Query OK, 3 rows affected (0.10 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM R3;
+-----+------+
| D   | E    |
+-----+------+
| 100 |  500 |
| 200 |  600 |
| 300 |  700 |
+-----+------+
3 rows in set (0.00 sec)

mysql> SELECT R1.A, R1.B, R1.C, R2.D, R3.E FROM R1 JOIN R2 ON R1.B = R2.B AND R1.C = R2.C JOIN R3 ON R2.D = R3.D;
+---+------+------+------+------+
| A | B    | C    | D    | E    |
+---+------+------+------+------+
| 1 |   10 |   20 |  100 |  500 |
| 2 |   30 |   40 |  200 |  600 |
| 3 |   50 |   60 |  300 |  700 |
+---+------+------+------+------+
3 rows in set (0.13 sec)
```

### 1. Observation

The original relation R(A,B,C,D,E) was analyzed using the given functional dependencies:

A → BC

BC → D

D → E

The attribute closure of A is:

$A^+ = \{A, B, C, D, E\}$

Therefore, A is a candidate key.

**2. Decomposition**
The relation was decomposed as:
R1(A, B, C)
R2(B, C, D)
R3(D, E)
The decomposed tables were successfully created and populated using SQL.

**3. Verification**
The decomposed tables were joined using the JOIN operation. The final output reproduced the original attributes:

A B C D E

1 10 20 100 500

2 30 40 200 600

3 50 60 300 700

**4. Justification**
Since A is a single-attribute candidate key, there is no partial dependency in the given relation. The decomposition separates the functional dependencies A → BC, BC → D, and D → E into individual relations.

5. Result
Thus, the given relation was successfully analyzed and decomposed based on the specified functional dependencies, and the original information was successfully reconstructed using SQL JOIN operations.

## 4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

## ANS:

```
mysql> use dbms;
Database changed
mysql> CREATE TABLE Employee ( EmpID INT PRIMARY KEY, EmpName VARCHAR(50), DeptID INT, DeptName VARCHAR(50), ManagerID INT );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Employee VALUES (101, 'Arun', 10, 'Computer Science', 501), (102, 'Priya', 20, 'Information Technology', 502), (103, 'Rahul', 10, 'Comput
er Science', 501);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Employee;
+-------+---------+--------+-----------------------+-----------+
| EmpID | EmpName | DeptID | DeptName              | ManagerID |
+-------+---------+--------+-----------------------+-----------+
|   101 | Arun    |     10 | Computer Science      |       501 |
|   102 | Priya   |     20 | Information Technology |       502 |
|   103 | Rahul   |     10 | Computer Science      |       501 |
+-------+---------+--------+-----------------------+-----------+
3 rows in set (0.00 sec)

mysql> CREATE TABLE Employee_3NF ( EmpID INT PRIMARY KEY, EmpName VARCHAR(50), DeptID INT );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Department ( DeptID INT PRIMARY KEY, DeptName VARCHAR(50), ManagerID INT );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Employee_3NF VALUES (101, 'Arun', 10), (102, 'Priya', 20), (103, 'Rahul', 10);
Query OK, 3 rows affected (0.14 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Department VALUES (10, 'Computer Science', 501), (20, 'Information Technology', 502);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Employee_3NF;
+-------+---------+--------+
| EmpID | EmpName | DeptID |
+-------+---------+--------+
|   101 | Arun    |     10 |
|   102 | Priya   |     20 |
|   103 | Rahul   |     10 |
+-------+---------+--------+
3 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM Employee_3NF;
+-------+---------+--------+
| EmpID | EmpName | DeptID |
+-------+---------+--------+
|   101 | Arun    |     10 |
|   102 | Priya   |     20 |
|   103 | Rahul   |     10 |
+-------+---------+--------+
3 rows in set (0.01 sec)

mysql> SELECT * FROM Department;
+--------+------------------------+-----------+
| DeptID | DeptName               | ManagerID |
+--------+------------------------+-----------+
|     10 | Computer Science       |       501 |
|     20 | Information Technology |       502 |
+--------+------------------------+-----------+
2 rows in set (0.00 sec)

mysql> SELECT E.EmpID, E.EmpName, E.DeptID, D.DeptName, D.ManagerID FROM Employee_3NF E JOIN Department D ON E.DeptID = D.DeptID;
+-------+---------+--------+------------------------+-----------+
| EmpID | EmpName | DeptID | DeptName               | ManagerID |
+-------+---------+--------+------------------------+-----------+
|   101 | Arun    |     10 | Computer Science       |       501 |
|   102 | Priya   |     20 | Information Technology |       502 |
|   103 | Rahul   |     10 | Computer Science       |       501 |
+-------+---------+--------+------------------------+-----------+
3 rows in set (0.01 sec)
```

**Justification:**
The original relation contained a transitive dependency EmpID → DeptID → DeptName, ManagerID. To remove this transitive dependency, the relation was decomposed into Employee_3NF(EmpID, EmpName, DeptID) and Department(DeptID, DeptName, ManagerID). The two tables are connected using DeptID.

**Result:**
Thus, the EMPLOYEE relation was successfully decomposed into 3NF by eliminating the transitive dependency, and the original information was successfully retrieved using a JOIN operation.

## 5.Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.

## ANS:

**Check whether R(A,B,C,D) is in BCNF. If not,decompose it.**

**Given Relation**

[
R(A,B,C,D)
]
Functional Dependencies:
[
AB\rightarrow C
]
[
C\rightarrow D
]
[
D\rightarrow A
]

**Find Candidate Keys**

**Find (AB^+)**

[
AB^+={A,B}
]
Using:
[
AB\rightarrow C

]
[
AB^+={A,B,C}
]
Using:
[
C\rightarrow D
]
[
\boxed{AB^+={A,B,C,D}}
]
So, **AB is a candidate key**.
Similarly:
[
BC^+={A,B,C,D}
]
So **BC is also a candidate key**.
And:
[
BD^+={A,B,C,D}
]
So **BD is also a candidate key**.
Therefore:
[
\boxed{Candidate\ Keys=AB,\ BC,\ BD}
]

**Check BCNF Condition**
**BCNF Rule**
For every functional dependency:
[
X\rightarrow Y
]
**X must be a superkey.**
Check the given FDs:

| FD | Is determinant a superkey? | Result |
|---|---|---|
| (AB\rightarrow C) | Yes | ✓ |
| (C\rightarrow D) | No | ✗ |
| (D\rightarrow A) | No | ✗ |

Therefore:
[
\boxed{R\text{ is NOT in BCNF}}
]
**Decompose the Relation**
Take the violating FD:
[
C\rightarrow D
]
Create the first relation:
[
\boxed{R_1(C,D)}
]
The remaining attributes form:

[
\boxed{R_2(A,B,C)}
]
Therefore:
[
\boxed{R(A,B,C,D)\rightarrow R_1(C,D)+R_2(A,B,C)}
]
**Check (R_1(C,D))**
FD:
[
C\rightarrow D
]
Here, C determines all attributes of (R_1).
Therefore, C is a key.
[
\boxed{R_1(C,D)\text{ is in BCNF}}
]
**Check (R_2(A,B,C))**
FD:
[
AB\rightarrow C
]
Here, AB is a key of (R_2).
Therefore:
[
\boxed{R_2(A,B,C)\text{ is in BCNF}}
]
**Final Answer**
[
\boxed{R_1(C,D)}
]
[
\boxed{R_2(A,B,C)}
]
Thus, the original relation **R is not in BCNF**, because (C\rightarrow D) and (D\rightarrow A) violate the BCNF condition. After decomposition, both relations are in **BCNF**.

## 6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

## ANS:

**Given:**
Relation:
R(A,B,C)
Functional Dependency:
$A \rightarrow B$
Decomposition:
R1(A,B),R2(A,C)
We need to verify whether the decomposition is **lossless-join** using the **Chase Algorithm**.

**Create the Chase Table**
Write all attributes of the original relation as columns.

| Relation | A | B | C |
|---|---|---|---|
| R1(A,B) | a | b | $c_1$ |
| R2(A,C) | a | $b_2$ | c |

Here:

- a, b, c are distinguished symbols.
- b2, c1 are different symbols.

**Apply the Functional Dependency**

Given:

$A \rightarrow B$

This means:

If two rows have the same value of A, their B values must also be the same.

**Compare the A Column**

Both rows have:

$A = a$

Therefore, according to:

$A \rightarrow B$

the B values must be equal.

Currently:

$b1 = b2$

So change:

$b2 \rightarrow b$

**Update the Chase Table**

| Relation | A | B | C |
|---|---|---|---|
| R1 | a | b | $c_1$ |
| R2 | a | b | c |

**Check the Chase Result**

The Chase process has successfully enforced the functional dependency:

$A \rightarrow B$

The common values are propagated correctly between the decomposed relations.

Therefore, the decomposition does not lose information.

Decomposition is Lossless-Join

**Final Conclusion**

The decomposition:

$R(A,B,C) \rightarrow R1(A,B), R2(A,C)$

is **lossless** because the Chase algorithm succeeds in satisfying the functional dependency $A \rightarrow B$.

## 7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

**ANS**:

```
mysql> CREATE TABLE TEACHER (
    ->     TID INT,
    ->     Subject VARCHAR(50),
    ->     Hobby VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER
    -> VALUES
    -> (101, 'DBMS', 'Reading'),
    -> (101, 'DBMS', 'Music'),
    -> (101, 'Java', 'Reading'),
    -> (101, 'Java', 'Music'),
    -> (102, 'Python', 'Cricket'),
    -> (102, 'Python', 'Reading');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER;
+------+---------+---------+
| TID  | Subject | Hobby   |
+------+---------+---------+
|  101 | DBMS    | Reading |
|  101 | DBMS    | Music   |
|  101 | Java    | Reading |
|  101 | Java    | Music   |
|  102 | Python  | Cricket |
|  102 | Python  | Reading |
+------+---------+---------+
```

```
mysql> CREATE TABLE TEACHER_SUBJECT (
    ->     TID INT,
    ->     Subject VARCHAR(50),
    ->     PRIMARY KEY (TID, Subject)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE TEACHER_HOBBY (
    ->     TID INT,
    ->     Hobby VARCHAR(50),
    ->     PRIMARY KEY (TID, Hobby)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO TEACHER_SUBJECT
    -> VALUES
    -> (101, 'DBMS'),
    -> (101, 'Java'),
    -> (102, 'Python');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM TEACHER_SUBJECT;
+-----+---------+
| TID | Subject |
+-----+---------+
| 101 | DBMS    |
| 101 | Java    |
| 102 | Python  |
+-----+---------+
3 rows in set (0.00 sec)

mysql> INSERT INTO TEACHER_HOBBY
    -> VALUES
    -> (101, 'Reading'),
    -> (101, 'Music'),
    -> (102, 'Cricket'),
    -> (102, 'Reading');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER_HOBBY;
+-----+---------+
| TID | Hobby   |
+-----+---------+
| 101 | Music   |
| 101 | Reading |
| 102 | Cricket |
| 102 | Reading |
+-----+---------+
4 rows in set (0.00 sec)
```

```
mysql> SELECT
    ->     TS.TID,
    ->     TS.Subject,
    ->     TH.Hobby
    -> FROM TEACHER_SUBJECT TS
    -> JOIN TEACHER_HOBBY TH
    -> ON TS.TID = TH.TID;
+-----+---------+---------+
| TID | Subject | Hobby   |
+-----+---------+---------+
| 101 | DBMS    | Music   |
| 101 | DBMS    | Reading |
| 101 | Java    | Music   |
| 101 | Java    | Reading |
| 102 | Python  | Cricket |
| 102 | Python  | Reading |
+-----+---------+---------+
6 rows in set (0.00 sec)
```

**Justification:**
The original TEACHER(TID, Subject, Hobby) relation contains two independent multivalued attributes, Subject and Hobby. Therefore, the multivalued dependencies are TID →→ Subject and TID →→ Hobby. Since TID is not a superkey for the original relation, it violates 4NF. The relation is decomposed into Teacher_Subject(TID, Subject) and Teacher_Hobby(TID, Hobby), eliminating the multivalued dependency redundancy.

**Result**
Thus, the multivalued dependencies in the TEACHER relation were identified and the relation was successfully decomposed into 4NF using SQL.

## 8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

## ANS:

**Given relation**
Consider:
SPJ(Supplier,Part,Project)
We need to show how a **join dependency causes redundancy** and decompose the relation into **5NF (Project-Join Normal Form)**.

**Understand the Relation**
Suppose a supplier can:

- Supply a particular part.
- Work on a particular project.
- A particular part can be used in a particular project.

Example:

| Supplier | Part | Project |
|---|---|---|
| S1 | P1 | J1 |
| S1 | P2 | J1 |
| S2 | P1 | J1 |

**Identify the Redundancy**
The three attributes contain separate relationships:
Supplier−Part Supplier−Project Part−Project

If these relationships are stored together in one table, combinations may be repeated.
For example, information about:
S1 supplies P1
S1 works on J1
P1 is used in J1
may be stored repeatedly as part of different combinations.
Therefore:
Join dependency can cause redundancy

**Identify the Join Dependency**
The original relation can be represented using three smaller relations:
SP(Supplier,Part) SJ(Supplier,Project) PJ(Part,Project)
The original relation can then be reconstructed by joining them:
SPJ=SP⋈SJ⋈PJ
This is the **join dependency**.

**Decompose the Relation**
**Relation 1**
SP(Supplier,Part)
**Relation 2**
SJ(Supplier,Project)
**Relation 3**
PJ(Part,Project)

**Why This Gives 5NF**
5NF requires that every non-trivial **join dependency** is implied by the candidate keys.
The decomposition separates the independent relationships and avoids unnecessary redundancy.
Thus:
SPJ→SP, SJ, PJ
and the original relation can be reconstructed by joining the three relations.

**Original relation:**
SPJ(Supplier,Part,Project)
**Join dependency:**
SPJ=SP⋈SJ⋈PJ
**5NF Decomposition:**
SP(Supplier,Part) SJ(Supplier,Project) PJ(Part,Project)
**Conclusion:**
The original relation may contain **redundant combinations** because three independent relationships are stored together. Decomposing it into SP, SJ, and PJ removes the redundancy and gives the required **5NF (Project-Join Normal Form)** structure.

## 9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.
## ANS:

```
mysql> CREATE TABLE COLLEGE (
    ->     StudentID INT,
    ->     StudentName VARCHAR(50),
    ->     DeptID INT,
    ->     DeptName VARCHAR(50),
    ->     CourseID INT,
    ->     CourseName VARCHAR(50),
    ->     InstructorID INT,
    ->     InstructorName VARCHAR(50),
    ->     Grade VARCHAR(5)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO COLLEGE
    -> VALUES
    -> (101, 'Arun', 10, 'CSE', 201, 'DBMS', 501, 'Kumar', 'A'),
    -> (101, 'Arun', 10, 'CSE', 202, 'Java', 502, 'Priya', 'B'),
    -> (102, 'Ravi', 10, 'CSE', 201, 'DBMS', 501, 'Kumar', 'A'),
    -> (103, 'Meena', 20, 'ECE', 203, 'Networks', 503, 'Raj', 'A'),
    -> (104, 'Anu', 20, 'ECE', 204, 'Python', 504, 'Divya', 'B');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM COLLEGE;
+-----------+-------------+--------+----------+----------+------------+--------------+----------------+-------+
| StudentID | StudentName | DeptID | DeptName | CourseID | CourseName | InstructorID | InstructorName | Grade |
+-----------+-------------+--------+----------+----------+------------+--------------+----------------+-------+
|       101 | Arun        |     10 | CSE      |      201 | DBMS       |          501 | Kumar          | A     |
|       101 | Arun        |     10 | CSE      |      202 | Java       |          502 | Priya          | B     |
|       102 | Ravi        |     10 | CSE      |      201 | DBMS       |          501 | Kumar          | A     |
|       103 | Meena       |     20 | ECE      |      203 | Networks   |          503 | Raj            | A     |
|       104 | Anu         |     20 | ECE      |      204 | Python     |          504 | Divya          | B     |
+-----------+-------------+--------+----------+----------+------------+--------------+----------------+-------+
```

```
mysql> UPDATE COLLEGE
    -> SET DeptName = 'Computer Science and Engineering'
    -> WHERE DeptID = 10;
Query OK, 3 rows affected (0.01 sec)
Rows matched: 3  Changed: 3  Warnings: 0

mysql> DELETE FROM COLLEGE
    -> WHERE StudentID = 104;
Query OK, 1 row affected (0.01 sec)

mysql> CREATE TABLE STUDENT (
    ->     StudentID INT PRIMARY KEY,
    ->     StudentName VARCHAR(50),
    ->     DeptID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO STUDENT
    -> VALUES
    -> (101, 'Arun', 10),
    -> (102, 'Ravi', 10),
    -> (103, 'Meena', 20),
    -> (104, 'Anu', 20);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT;
+-----------+-------------+--------+
| StudentID | StudentName | DeptID |
+-----------+-------------+--------+
|       101 | Arun        |     10 |
|       102 | Ravi        |     10 |
|       103 | Meena       |     20 |
|       104 | Anu         |     20 |
+-----------+-------------+--------+
4 rows in set (0.00 sec)

mysql> CREATE TABLE COURSE (
    ->     CourseID INT PRIMARY KEY,
    ->     CourseName VARCHAR(50),
    ->     InstructorID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO COURSE
    -> VALUES
    -> (201, 'DBMS', 501),
    -> (202, 'Java', 502),
    -> (203, 'Networks', 503),
    -> (204, 'Python', 504);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE ENROLLMENT (
    ->     StudentID INT,
    ->     CourseID INT,
    ->     Grade VARCHAR(5),
    ->     PRIMARY KEY (StudentID, CourseID)
    ->     PRIMARY KEY (StudentID, CourseID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO ENROLLMENT
    -> VALUES
    -> (101, 201, 'A'),
    -> (101, 202, 'B'),
    -> (102, 201, 'A'),
    -> (103, 203, 'A'),
    -> (104, 204, 'B');
Query OK, 5 rows affected (0.01 sec)
```

```
mysql> CREATE TABLE DEPARTMENT (
    ->     DeptID INT PRIMARY KEY,
    ->     DeptName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO DEPARTMENT
    -> VALUES
    -> (10, 'CSE'),
    -> (20, 'ECE');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE INSTRUCTOR (
    ->     InstructorID INT PRIMARY KEY,
    ->     InstructorName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO INSTRUCTOR
    -> VALUES
    -> (501, 'Kumar'),
    -> (502, 'Priya'),
    -> (503, 'Raj'),
    -> (504, 'Divya');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE STUDENT
    -> ADD FOREIGN KEY (DeptID)
    -> REFERENCES DEPARTMENT(DeptID);
```

```
mysql> ALTER TABLE STUDENT
    -> ADD FOREIGN KEY (DeptID)
    -> REFERENCES DEPARTMENT(DeptID);
Query OK, 4 rows affected (0.12 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE COURSE
    -> ADD FOREIGN KEY (InstructorID)
    -> REFERENCES INSTRUCTOR(InstructorID);
Query OK, 4 rows affected (0.11 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE ENROLLMENT
    -> ADD FOREIGN KEY (StudentID)
    -> REFERENCES STUDENT(StudentID);
Query OK, 5 rows affected (0.07 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE ENROLLMENT
    -> ADD FOREIGN KEY (CourseID)
    -> REFERENCES COURSE(CourseID);
Query OK, 5 rows affected (0.13 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT
    ->     S.StudentID,
    ->     S.StudentName,
    ->     D.DeptName,
    ->     C.CourseID,
    ->     C.CourseName,
    ->     I.InstructorName,
    ->     E.Grade
   ->     I.InstructorName,
   ->     E.Grade
   -> FROM ENROLLMENT E
   -> JOIN STUDENT S
   ->     ON E.StudentID = S.StudentID
   -> JOIN DEPARTMENT D
   ->     ON S.DeptID = D.DeptID
   -> JOIN COURSE C
   ->     ON E.CourseID = C.CourseID
   -> JOIN INSTRUCTOR I
   ->     ON C.InstructorID = I.InstructorID;
+-----------+-------------+----------+----------+------------+----------------+-------+
| StudentID | StudentName | DeptName | CourseID | CourseName | InstructorName | Grade |
+-----------+-------------+----------+----------+------------+----------------+-------+
|       101 | Arun        | CSE      |      201 | DBMS       | Kumar          | A     |
|       101 | Arun        | CSE      |      202 | Java       | Priya          | B     |
|       102 | Ravi        | CSE      |      201 | DBMS       | Kumar          | A     |
|       103 | Meena       | ECE      |      203 | Networks   | Raj            | A     |
|       104 | Anu         | ECE      |      204 | Python     | Divya          | B     |
+-----------+-------------+----------+----------+------------+----------------+-------+
5 rows in set (0.01 sec)

mysql> UPDATE DEPARTMENT
    -> SET DeptName = 'Computer Science'
    -> WHERE DeptID = 10;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> UPDATE DEPARTMENT
    -> SET DeptName = 'Computer Science'
    -> WHERE DeptID = 10;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from department;
+--------+------------------+
| DeptID | DeptName         |
+--------+------------------+
|     10 | Computer Science |
|     20 | ECE              |
+--------+------------------+
2 rows in set (0.00 sec)
```

**Justification:**
The original College table contained redundant student, course, and faculty information. This resulted in update, insertion, and deletion anomalies. To remove these anomalies, the relation was decomposed into Student, Course, and Enrollment tables. Each table stores information about a single entity, and relationships are maintained using primary and foreign keys. The final tables satisfy the requirements of 3NF.

**Result:**
Thus, the insertion, deletion, and update anomalies in the poorly designed college database were identified and the relation was successfully normalized into 3NF using SQL**.**

## 10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

## ANS:

**Attribute Closure and Candidate Keys**
Using the **same relation and FDs given in Q3**:

R(A,B,C,D,E)

Functional Dependencies:

A→BC BC→D D→E

We need to **find attribute closures and determine the candidate key**.

**Find A+**
Start with:

A+={A}

Using:

A→BC

add B and C:

A+={A,B,C}

Using:

BC→D

add D:

A+={A,B,C,D}

Using:

D→E

add E:

A+={A,B,C,D,E}

Since A+ contains **all attributes**, A is a key.

**Find B+**
Start:

B+={B}

No FD can be applied.

B+={B}

So B is **not a key**.

**Find C+**

Start:

C+={C}

No FD can be applied.

C+={C}

So C is **not a key**.

**Find D+**

Start:

D+={D}

Using:

D→E

we get:

D+={D,E}

So D is **not a key**.

**Find E+**

Start:

E+={E}

No FD can be applied.

E+={E}

So E is **not a key**.

**Determine Candidate Key**

Only A gives all attributes:

A+={A,B,C,D,E}

Therefore:

A is the candidate key

**Final Answer**

| Attribute Set | Closure | Key? |
|---|---|---|
| A+ | {A,B,C,D,E} | **Yes** |
| B+ | {B} | No |
| C+ | {C} | No |
| D+ | {D,E} | No |
| E+ | {E} | No |

**Candidate Key:**

{A}